

Relatório Técnico

Mini SOAR de Integração de Ferramentas de Threat Intelligence

cenourin

Portfólio técnico — IA aplicada a Cibersegurança

15 de julho de 2026

Abstract

Este documento descreve os requisitos, a arquitetura, o modelo de dados, o plano e a execução de testes, a análise dos resultados obtidos e uma proposta de aplicação em cenário real do *Mini SOAR de Integração de Ferramentas*. O sistema recebe um indicador suspeito (IP ou hash de arquivo), consulta automaticamente sua reputação em uma fonte de threat intelligence e envia um alerta formatado para um canal de notificação, reproduzindo em pequena escala um playbook de SOAR (*Security Orchestration, Automation and Response*).

Contents

1	Introdução e contexto	1
2	Documento de requisitos	1
2.1	Escopo	1
2.2	Atores	2
2.3	Requisitos funcionais	2
2.4	Requisitos não funcionais	2
2.5	Casos de uso	3
2.6	CrITÉrios de aceite	3
3	Arquitetura e engenharia de software	3
3.1	Visão geral	3
3.2	Módulos	3
3.3	Decisões de arquitetura	4
3.4	Modelo de dados	4
3.5	Contrato da API (resumo)	5
3.6	Empacotamento e execução	5
4	Explicação dos dados	6
4.1	Base local de reputação (<code>mock_reputation.json</code>)	6
4.2	Fila de indicadores (<code>indicators_queue.json</code>)	6
4.3	Persistência incremental e idempotência	6
5	Plano e execução de testes	6
5.1	Estratégia	6
5.2	Casos de teste	7
5.3	Execução real	7
5.4	Cobertura de código	8
6	Análise de resultados	8

7	Aplicação em um caso real	9
7.1	Do protótipo a um playbook de produção	9
7.2	Custo e limites de taxa	10
7.3	Extensões naturais	10
8	Conclusão	10

1 Introdução e contexto

O projeto foi motivado pelo interesse pessoal em criar scripts e automações para otimização de processos de um SOC e explorar a integração entre ferramentas de segurança. O objetivo é demonstrar, em escala reduzida porém funcional, a automação de um fluxo típico de resposta a incidentes: receber um indicador de comprometimento (IoC), consultar sua reputação e notificar um canal de resposta quando ele for malicioso — sem exigir bloqueio automático, preservando a decisão final com um analista humano.

Assim como o Projeto 1 deste portfólio (`projeto-1-agente-triagem-logs`), este sistema segue duas premissas centrais de engenharia:

1. **Disponibilidade sem dependência externa obrigatória:** o serviço deve funcionar integralmente sem exigir chaves de API pagas nem um workspace de Slack/Discord real.
2. **Determinismo em testes:** a suíte automatizada não deve depender de rede externa.

Essas premissas motivaram duas decisões de arquitetura equivalentes: motor duplo de threat intelligence (real + mock) e motor duplo de notificação (webhook real + console), descritas na Seção 3.

2 Documento de requisitos

2.1 Escopo

Dentro do escopo: consulta de reputação sob demanda via API; detecção automática do tipo de indicador (IPv4 ou hash MD5/SHA1/SHA256); envio de alerta formatado para Slack/Discord quando malicioso; execução de um *playbook* em lote a partir de uma fila de indicadores, disparável sob demanda ou por um worker em intervalo fixo; histórico consultável de consultas e notificações; modo real (chaves configuradas) ou modo mock (base local de reputação).

Fora do escopo: bloqueio automático do indicador em firewall/proxy real; autenticação/autorização de usuários da API; suporte a outros tipos de indicador (domínio, URL) além de IP e hash de arquivo.

2.2 Atores

- **Analista de SOC** — envia indicadores suspeitos para verificação e recebe alertas.
- **Fonte de threat intelligence** (AbuseIPDB / VirusTotal) — serviço externo consultado.
- **Canal de notificação** (Slack ou Discord via webhook) — destino dos alertas.
- **Worker agendado** — processa a fila de indicadores pendentes periodicamente.

2.3 Requisitos funcionais

ID	Descrição	Prioridade
RF01	Receber um indicador (IP ou hash) via <code>POST /lookup</code> e retornar sua reputação.	Alta
RF02	Detectar automaticamente se o indicador é IPv4 ou hash e escolher a fonte adequada.	Alta
RF03	Quando malicioso, enviar alerta formatado ao webhook configurado.	Alta
RF04	Persistir todas as consultas (<code>GET /lookups</code> , <code>GET /lookups/{id}</code>).	Alta
RF05	Persistir todas as notificações emitidas, com status de entrega (<code>GET /notifications</code>).	Média
RF06	Processar fila de indicadores pendentes via <code>POST /playbook/run-queue</code> .	Alta
RF07	Worker deve reexecutar o playbook automaticamente em intervalo configurável.	Média
RF08	Sem chaves de threat intel, usar base local de reputação mockada sem falhar.	Alta
RF09	Sem webhook configurado, registrar o alerta localmente (modo mock) sem falhar.	Alta
RF10	Expor <code>GET /health</code> para checagem de disponibilidade.	Alta

Table 1: Requisitos funcionais

2.4 Requisitos não funcionais

ID	Descrição
RNF01	O serviço deve subir integralmente via <code>docker compose up</code> (API + worker) sem configuração manual adicional.
RNF02	Falha na fonte real de threat intel deve degradar automaticamente para mock, nunca 500.
RNF03	Falha ao entregar o webhook não interrompe o fluxo; a consulta é sempre retornada e persistida.
RNF04	Nenhuma credencial versionada em texto puro; apenas <code>.env.example</code> é commitado.
RNF05	Cobertura de testes automatizados sem dependência de rede real.

Table 2: Requisitos não funcionais

2.5 Casos de uso

UC01 — Consultar reputação de um indicador. Analista envia `POST /lookup`; o sistema detecta o tipo, consulta a fonte adequada (real ou mock), persiste o resultado e, se malicioso, formata e envia (ou registra) o alerta.

UC02 — Executar playbook em lote. Operador (ou worker) chama `POST /playbook/run-queue`; o sistema lê a fila de indicadores e aplica UC01 a cada um ainda não consultado, retornando um resumo agregado.

UC03 — Automação periódica (worker). O serviço `worker` aguarda o intervalo configurado, chama o playbook da API, registra o resumo no log e repete indefinidamente.

2.6 Critérios de aceite

- `docker compose up --build` inicia API (porta 8002) e worker sem erros.
- `curl http://localhost:8002/health` retorna 200 OK.
- Um IP malicioso conhecido da base mock retorna `malicious: true` e gera entrada em `GET /notifications`.
- `POST /playbook/run-queue` processa toda a fila de exemplo e retorna contadores consistentes com `GET /lookups`.
- `pytest` roda com sucesso, sem acessar rede externa.

3 Arquitetura e engenharia de software

3.1 Visão geral

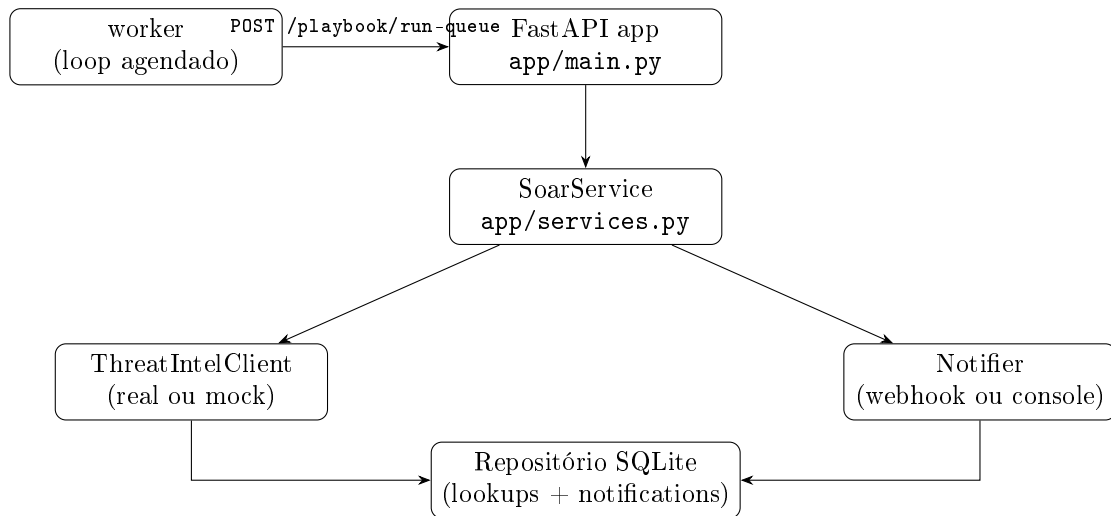


Figure 1: Visão geral: o **worker** chama a API periodicamente; a API delega ao **SoarService**, que consulta reputação, notifica e persiste.

3.2 Módulos

Módulo	Responsabilidade
<code>app/main.py</code>	API FastAPI: rotas de lookup, histórico e execução do playbook.
<code>app/services.py</code>	SoarService : orquestra detecção de tipo, consulta, notificação e persistência (UC01).
<code>app/indicator.py</code>	Detecção e validação do tipo de indicador (<code>ip</code> ou <code>hash</code>).
<code>app/threat_intel_client.py</code>	Consulta AbuseIPDB (IP) ou VirusTotal (hash); fallback para <code>MockReputationSource</code> .
<code>app/notifier.py</code>	Envia alerta para Slack/Discord; fallback console.
<code>app/db.py</code>	Acesso a dados via SQLite: tabelas <code>lookups</code> e <code>notifications</code> .
<code>app/queue_runner.py</code>	Leitura da fila e execução em lote (UC02), reutilizado pela API e pelo worker.
<code>worker/loop.py</code>	Processo de longa duração que chama a API em intervalo fixo (UC03), em container separado.

Table 3: Módulos do sistema e responsabilidades

3.3 Decisões de arquitetura

3.1 — Motor duplo para threat intel (real + mock). *Contexto:* o objetivo é consultar automaticamente a reputação em uma API de threat intel, mas o projeto precisa ficar demonstrável sem exigir chaves pagas. *Decisão:* `ThreatIntelClient` tenta a API real (AbuseIPDB para IP, VirusTotal para hash) apenas se a respectiva chave estiver definida; qualquer ausência de chave ou exceção HTTP aciona `MockReputationSource`, que consulta `data/mock_reputation.json` — incluindo o hash de teste EICAR, indicador padrão da indústria para simular malware sem risco real. O campo `source` da resposta indica qual foi usado.

3.2 — Motor duplo para notificação (webhook real + console/mock). *Decisão:* `Notifier` publica no webhook configurado (Slack tem prioridade se ambos estiverem presentes); se nenhum estiver configurado, ou se o POST falhar, o alerta é registrado com `channel="console"` e `delivered=false`, sem interromper o fluxo (RNF03).

3.3 — Worker como container separado em vez de cron na imagem da API. *Contexto:* o cenário de referência é rodar como script agendado em Linux (cron). *Decisão:* em vez de instalar `cron` dentro do container da API, o agendamento é um segundo serviço Docker com um laço Python simples (`while True: run(); sleep(N)`) que chama a API via HTTP — funcionalmente equivalente a um cron job, porém mais simples de operar e observar em Docker Compose (`docker compose logs worker`) e mais fácil de testar isoladamente.

3.4 Modelo de dados

Coluna (lookups)	Tipo	Descrição
<code>id</code>	INTEGER PK	Identificador autoincremental
<code>indicator</code>	TEXT	IP ou hash consultado
<code>indicator_type</code>	TEXT	<code>ip</code> ou <code>hash</code>
<code>malicious</code>	INTEGER (bool)	Resultado da consulta
<code>score</code>	INTEGER	Score de confiança/abuso (0–100)
<code>source</code>	TEXT	<code>abuseipdb</code> / <code>virustotal</code> / <code>mock</code>
<code>categories</code>	TEXT (JSON)	Categorias de ameaça retornadas
<code>created_at</code>	TEXT (ISO 8601)	Timestamp da consulta

Table 4: Esquema da tabela `lookups`

Coluna (notifications)	Tipo	Descrição
<code>id</code>	INTEGER PK	Identificador autoincremental
<code>lookup_id</code>	INTEGER FK	Referência à consulta que originou o alerta
<code>indicator</code>	TEXT	Indicador associado
<code>channel</code>	TEXT	<code>slack</code> / <code>discord</code> / <code>console</code>
<code>message</code>	TEXT	Corpo do alerta enviado
<code>delivered</code>	INTEGER (bool)	Se o webhook respondeu com sucesso

Coluna (notifications)	Tipo	Descrição
created_at	TEXT (ISO 8601)	Timestamp do envio

Table 5: Esquema da tabela notifications

3.5 Contrato da API (resumo)

```
POST /lookup
// request
{ "indicator": "203.0.113.9" }

// response 200
{
  "lookup": {
    "id": 1, "indicator": "203.0.113.9", "indicator_type": "ip",
    "malicious": true, "score": 97, "source": "mock",
    "categories": ["ssh-bruteforce", "scanning"], "created_at": "2026-07-15T12:00:00"
  },
  "notification": {
    "id": 1, "channel": "console", "delivered": false,
    "message": "Indicador malicioso detectado: 203.0.113.9 (score 97, ssh-bruteforce, scanning)"
  }
}
```

POST /playbook/run-queue → {"processed": 4, "malicious": 2, "notifications_sent": 2}.

3.6 Empacotamento e execução

Imagem base python:3.12-slim, compartilhada entre API e worker (mesmo requirements.txt, CMD diferente). O docker-compose.yml define dois serviços: api (porta 8002) e worker (sem porta exposta, comunica-se com api pela rede interna do Compose via http://api:8002), com dependência de healthcheck entre eles. Volume nomeado persiste o SQLite; ./data é montado somente leitura para os arquivos de exemplo.

4 Explicação dos dados

4.1 Base local de reputação (mock_reputation.json)

Simula a resposta de um provedor de threat intelligence real. Contém cinco entradas: quatro indicadores marcados como maliciosos (três IPs e um hash) e um IP benigno de referência (8.8.8.8, o resolvedor DNS público da Google, usado deliberadamente como caso de controle negativo por ser um endereço amplamente conhecido e inequivocamente não malicioso).

Indicador	Tipo	Score	Categorias
203.0.113.9	IP	97	ssh-bruteforce, scanning
198.51.100.77	IP	88	port-scan
45.33.32.156	IP	76	c2-server
44d88612fea8af... (MD5)	hash	100	eicar-test-file

Indicador	Tipo	Score	Categorias
8.8.8.8	IP	0 (benigno)	—

Table 6: Conteúdo da base local de reputação

Os três IPs maliciosos pertencem às mesmas faixas reservadas para documentação usadas no Projeto 1 (RFC 5737, 203.0.113.0/24 e 198.51.100.0/24), reforçando a consistência do portfólio; o hash 44d88612fea8a8f36de82e1278abb02f é o hash MD5 do arquivo de teste EICAR, um padrão da indústria antivírus para simular a detecção de malware sem manipular um binário malicioso real.

4.2 Fila de indicadores (indicators_queue.json)

Lista de cinco indicadores pendentes de consulta, no formato [{"indicator": "..."}, ...], consumida por `POST /playbook/run-queue` e pelo `worker`. No dataset de exemplo, a fila referencia exatamente os quatro indicadores maliciosos da base mock mais o indicador benigno de controle — isto é, 100% dos indicadores cadastrados na base de reputação, permitindo observar os dois desfechos possíveis (malicioso → notificação; benigno → sem notificação) em uma única execução do `playbook`.

4.3 Persistência incremental e idempotência

A função `indicator_already_looked_up` evita reprocessar um indicador que já tenha sido consultado anteriormente, tornando `run_queue` idempotente entre execuções sucessivas do `worker`: apenas indicadores novos na fila geram nova consulta e, potencialmente, nova notificação. Isso é relevante em produção para não gerar alertas duplicados a cada ciclo do agendamento periódico.

5 Plano e execução de testes

5.1 Estratégia

Testes automatizados com `pytest`. Nenhum teste depende de rede externa: o `ThreatIntelClient` e o `Notifier` são exercitados em modo mock (sem chaves de API nem webhooks configurados), garantindo determinismo e reprodutibilidade (ver `tests/conftest.py`).

5.2 Casos de teste

ID	Caso	Tipo	Resultado esperado
T01	<code>test_detect_ipv4</code>	Unitário	"203.0.113.9" detectado como ip
T02	<code>test_detect_hash_md5</code>	Unitário	hash de 32 hex chars detectado como hash
T03	<code>test_detect_hash_sha256</code>	Unitário	hash de 64 hex chars detectado como hash
T04	<code>test_detect_invalid_indicator</code>	Unitário	string inválida levanta <code>InvalidIndicatorError</code>
T05	<code>test_mock_reputation_known_malicious_ip</code>	Unitário	IP malicioso mock retorna <code>malicious=True</code>
T06	<code>test_mock_reputation_unknown_indicator</code>	Unitário	indicador ausente retorna <code>malicious=False, score=0</code>

ID	Caso	Tipo	Resultado esperado
T07	test_mock_reputation_known_hash	Unitário	hash malicioso mock retorna indicator_type="hash"
T08	test_console_notifier_records_message	Unitário	sem webhook, channel="console", delivered=False
T09	test_format_alert_message_contains_indicator	Unitário	mensagem contém indicador e score
T10	test_health_endpoint	Integração	200
T11	test_lookup_malicious_indicator_triggers_notification	Integração	malicious=true + notificação criada
T12	test_lookup_benign_indicator_no_notification	Integração	sem notificação
T13	test_lookup_rejects_invalid_indicator	Integração	422
T14	test_playbook_run_queue	Integração	contadores coerentes com a fila de exemplo
T15	test_lookups_history_endpoint	Integração	histórico reflete as consultas
T16	test_get_lookup_not_found	Integração	404

Table 7: Casos de teste automatizados

5.3 Execução real

A suíte foi executada localmente com `pytest -v` em 15/07/2026, sem acesso à rede e sem credenciais configuradas:

```
===== test session starts =====
collected 16 items

tests/test_api.py::test_health_endpoint PASSED [ 6%]
tests/test_api.py::test_lookup_malicious_indicator_triggers_notification PASSED [ 12%]
tests/test_api.py::test_lookup_benign_indicator_no_notification PASSED [ 18%]
tests/test_api.py::test_lookup_rejects_invalid_indicator PASSED [ 25%]
tests/test_api.py::test_playbook_run_queue PASSED [ 31%]
tests/test_api.py::test_lookups_history_endpoint PASSED [ 37%]
tests/test_api.py::test_get_lookup_not_found PASSED [ 43%]
tests/test_indicator.py::test_detect_ipv4 PASSED [ 50%]
tests/test_indicator.py::test_detect_hash_md5 PASSED [ 56%]
tests/test_indicator.py::test_detect_hash_sha256 PASSED [ 62%]
tests/test_indicator.py::test_detect_invalid_indicator PASSED [ 68%]
tests/test_notifier.py::test_console_notifier_records_message PASSED [ 75%]
tests/test_notifier.py::test_format_alert_message_contains_indicator PASSED [ 81%]
tests/test_threat_intel.py::test_mock_reputation_known_malicious_ip PASSED [ 87%]
tests/test_threat_intel.py::test_mock_reputation_unknown_indicator PASSED [ 93%]
tests/test_threat_intel.py::test_mock_reputation_known_hash PASSED [100%]

===== 16 passed in 0.72s =====
```

Execução manual do playbook contra a fila de exemplo, fora da suíte de testes:

```
>>> run_queue()
{'processed': 5, 'malicious': 4, 'notifications_sent': 4}
```

Resultado coerente com o conteúdo da Seção 4: dos cinco indicadores da fila, quatro são maliciosos na base mock (gerando quatro notificações) e um é benigno (8.8.8.8, sem notificação).

5.4 Cobertura de código

Módulo	Linhas	Não cober- tas	Cobertura	Observação
app/config.py	14	0	100%	
app/indicator.py	11	0	100%	
app/models.py	42	0	100%	
app/services.py	16	0	100%	
app/db.py	63	2	97%	
app/queue_runner.py	31	2	94%	
app/main.py	41	3	93%	rota de erro 404 parcial- mente coberta
app/threat_intel_client.py	52	20	62%	caminhos reais AbuseIPDB/VirusTotal não exercitados
app/notifier.py	29	13	55%	caminhos de webhook real (Slack/Discord) não exercitados
Total	402	40	90%	

Table 8: Cobertura de código por módulo

6 Análise de resultados

Corretude funcional. As 16 execuções automatizadas passaram (100%), cobrindo detecção de indicador, consulta mock de reputação, notificação em modo console e os seis comportamentos de API (RF01, RF02, RF04–RF06, RF10). A execução manual do playbook sobre a fila de exemplo confirmou o comportamento agregado esperado: `processed=5`, `malicious=4`, `notifications_sent=4` — os quatro indicadores maliciosos da base mock geraram notificação, o indicador benigno não gerou.

Cobertura como indicador de risco. De forma simétrica ao Projeto 1, os módulos determinísticos (`indicator`, `services`, `models`, `config`) têm cobertura de 93–100%, enquanto os dois módulos que integram serviços externos reais (`threat_intel_client.py`, 62%, e `notifier.py`, 55%) têm cobertura sensivelmente menor — consequência direta da decisão de manter os testes livres de rede (RNF05). Os trechos não cobertos são precisamente os métodos `_lookup_abuseipdb`, `_lookup_virustotal` e `_post_webhook`, isto é, os adaptadores de I/O externo. Mitigação recomendada, não implementada neste protótipo: testes de contrato com `httpx.MockTransport` ou biblioteca equivalente, simulando as respostas HTTP da AbuseIPDB/VirusTotal/Slack sem round-trip de rede real.

Idempotência do playbook. O uso de `indicator_already_looked_up` evita notificações duplicadas em execuções repetidas do worker sobre a mesma fila — comportamento relevante quando o `PLAYBOOK_INTERVAL_SECONDS` é curto e a fila não é constantemente renovada; o teste T14 valida o processamento inicial, mas não há um teste automatizado dedicado a confirmar explicitamente a não duplicação em uma segunda chamada consecutiva de `run_queue` — uma lacuna identificada por esta análise e recomendada para uma próxima iteração da suíte.

Desempenho. Não foi realizado *benchmark* formal. Em modo mock, cada consulta é uma leitura de dicionário em memória (`MockReputationSource`), com custo desprezível; em modo real, cada consulta é uma chamada HTTP síncrona com timeout de 5 segundos configurado em `threat_intel_client.py` e em `notifier.py`, o que limita o tempo máximo de resposta por

indicador em caso de degradação do provedor externo, mas também significa que o processamento da fila em lote é serial — processar N indicadores reais pode levar até $N \times 5s$ no pior caso, já que `run_queue` não paraleliza as chamadas.

Limitações conhecidas. (i) suporte restrito a IPv4 e hash (sem IPv6, domínio ou URL); (ii) processamento serial da fila, sem paralelismo nem *backoff* exponencial em caso de *rate limit* da API real; (iii) sem testes de contrato para os adaptadores HTTP reais; (iv) sem autenticação na API, adequada apenas a um ambiente de rede interna confiável, não à exposição direta na internet.

7 Aplicação em um caso real

7.1 Do protótipo a um playbook de produção

O desenho como dois serviços (`api` e `worker`) já reflete um padrão de produção comum em automação de SOC: separar a lógica de negócio, exposta como API, do agendamento, que a invoca. Uma integração realista em um ambiente como o de um provedor de SOC gerenciado seguiria:

1. **Origem dos indicadores:** em vez de um arquivo estático (`indicators_queue.json`), a fila seria alimentada automaticamente por outras ferramentas do SOC — por exemplo, o Projeto 1 deste portfólio poderia publicar nesta fila todo IP extraído de um evento classificado como **SUSPEITO/CRITICO**, encadeando os dois projetos em um pipeline único de triagem → enriquecimento → notificação.
2. **Paralelismo e resiliência:** substituir o processamento serial de `run_queue` por um pool de workers concorrentes com *backoff* exponencial e respeito a limites de *rate limit* das APIs reais (AbuseIPDB e VirusTotal têm cotas agressivas nos planos gratuitos), e mover a fila de um arquivo JSON para uma fila de mensageria (SQS, RabbitMQ) com re-entrega e *dead-letter queue* para indicadores que falharem repetidamente.
3. **Enriquecimento adicional:** agregar mais fontes de threat intel (ex.: MISP, feeds internos do provedor de SOC) no mesmo padrão de `ThreatIntelClient` — a interface já desacoplada por `source` no modelo de dados permite adicionar novas fontes sem alterar o contrato da API.
4. **Fechamento do ciclo com o analista:** hoje a notificação é apenas informativa; em produção, o alerta enviado ao Slack/Discord incluiria um link direto para abrir um caso no sistema de tickets do SOC (ex.: criar registro em um SOAR comercial), e a decisão de bloquear ou não o indicador permaneceria com o analista — mantendo, tal como no Projeto 1, o princípio de *humano no laço* antes de qualquer ação com efeito real na rede.
5. **Autenticação:** adicionar autenticação de serviço (API key ou mTLS) entre `worker` e `api`, e entre o pipeline de origem e o `POST /lookup`, já que o protótipo atual assume uma rede interna confiável.

7.2 Custo e limites de taxa

As APIs gratuitas de AbuseIPDB e VirusTotal têm cotas diárias limitadas. Em um SOC com alto volume de indicadores, a estratégia de deduplicação já presente (`indicator_already_looked_up`) é o primeiro nível de economia de cota; como evolução, um cache com expiração (ex.: reconsultar o mesmo indicador no máximo uma vez por 24h, mesmo que ele reapareça em execuções futuras da fila) reduziria ainda mais o consumo de cota sem comprometer a atualidade da informação para o caso de uso de triagem.

7.3 Extensões naturais

Suporte a domínios e URLs (além de IP e hash) é a extensão mais natural, reaproveitando o mesmo padrão de `detect_indicator_type` com uma nova regra de detecção e um novo adaptador em `ThreatIntelClient` — sem alterar `SoarService` nem o contrato da API, graças ao desacoplamento já estabelecido na arquitetura.

8 Conclusão

O protótipo atende a todos os requisitos funcionais e não funcionais definidos, com 100% dos 16 casos de teste automatizados passando e 90% de cobertura de código global, e uma execução manual do playbook que reproduziu o comportamento esperado sobre o dataset de exemplo (4 de 5 indicadores maliciosos corretamente notificados). As decisões de motor duplo para threat intel e para notificação garantiram um sistema demonstrável offline e sem custo, com o mesmo *trade-off* de cobertura de teste mais baixa nos adaptadores de integração externa já discutido no relatório do Projeto 1. A Seção 7 detalha um caminho concreto de evolução para um playbook de produção, incluindo paralelismo, filas de mensageria e fechamento do ciclo com o analista de SOC.